

Efficient DeepSeek Sparse Attention Kernels

MLSys 2026 FlashInfer AI Kernel Generation Contest:

DSA Track, Agent-Human Collaboration Sub-Track Winner 🏆

Doğaç Eldenk
Northwestern University
United States
dogacel@gmail.com

Abstract

We describe our submission to the Agent-Human Collaboration sub-track of the MLSys 2026 FlashInfer AI Kernel Generation Contest on the DeepSeek Sparse Attention (DSA) track. The submitted kernels are the result of two complementary pipelines: an autonomous Claude Code optimization loop, and a human-led collaboration phase that bootstrapped the loop and contributed structural optimizations the loop did not find on its own. The human identified workload-level fast paths and pruned unreachable code paths; the agent produced the Triton sparse attention and FP8 scoring kernels and ran the final hyper-parameter sweeps. The shipped solution comprises a Triton sparse-attention kernel with split-K and an atomic-barrier combine, an FP8 importance-scoring kernel, and a CUDA radix top- K specialized to the workload’s expected input sizes. On the contest’s B200 evaluation, our kernels achieve 37.07x mean speedup over FlashInfer benchmarks and rank #1 on the leaderboard.

Keywords

sparse attention, Triton, CUDA, GPU kernels, coding agents, human-AI collaboration, DeepSeek, FlashInfer, MLSys

1 Introduction

DeepSeek Sparse Attention (DSA), introduced with DeepSeek-V3.2 [2], splits long-context attention into two stages: a low-rank “lightning indexer” selects per-query top- k key positions, and a sparse attention kernel attends only over those indices into a paged KV cache. The MLSys 2026 FlashInfer AI Kernel Generation Contest [1] asks for high-performance DSA kernels for the *Blackwell Architecture*, evaluated on *NVIDIA B200* against a reference FlashInfer implementation.

Our kernel competed in the *Agent-Human Collaboration* sub-track. We first ran a human-in-the-loop phase to bootstrap a credibly-optimized Triton kernel and to learn how the agent behaves under different prompts, tools, and constraints. We distilled those observations into an autonomous loop (described in our companion Full-Agent write-up) and ran it unattended. The loop, somewhat to our surprise, produced kernels measurably more optimized than the initial human-in-the-loop phase had ended on. We then merged the two: kept the autonomous loop’s Triton sparse attention and FP8 scoring, retained the collaboration phase’s CUDA FilteredTopK for the long-context indexer regime, and ran one final autonomous pass purely for hyper-parameter sweeps. This report focuses on the kernel and what the human contributed, most importantly the failure modes the human kept correcting.

2 Background

DSA decoding has two kernels in series. The **TopK indexer** computes per-token scores against an FP8 paged K-cache (page size 64, structure-of-arrays per-row scales) and returns the top $K=2048$ token indices, padded with -1 . The lightning indexer uses a single *shared-across-heads* key projection K_t (no head dimension) against $H=64$ per-head queries $q_{b,h}$, so the score is $I_{b,t} = \text{scale}_t \cdot \sum_h w_{b,h} \text{relu}(\langle q_{b,h}, K_t \rangle)$. The **sparse attention** kernel takes paged latent KV caches and the $K=2048$ sparse indices per token (with -1 padding) and returns flash-attention output and LSE. The TopK indexer is evaluated on 128 traces and Sparse Attention on 23 traces, sampled from real DeepSeek-V3.2 inference.

3 TopK Indexer

The solution uses a **two-page dispatch**:

- (1) **Passthrough** ($\text{max_seq_padded} \leq K$): Every valid token is in the top- K . Emit token IDs directly via page-table remap.
- (2) **Score + Select** ($\text{max_seq_padded} > K$): Fused FP8 kernel $\rightarrow$ flashinfer radix top- K .

3.1 Score Kernel: FP8 Importance Scoring

Each program covers one (b, p) pair: one batch, one logical KV page, 64 consecutive token positions. The score for batch b at *global* token position $t = 64p + r$ (page p , offset $r \in [0, 64)$) is

$$I_{b,t} = \text{scale}_{p,r} \cdot \sum_{j=1}^H w_b^{(j)} \cdot \max(0, \langle q_b^{(j)}, k_{p,r} \rangle) \quad (1)$$

where $\langle \cdot, \cdot \rangle$ is the $D=128$ inner product. The key vector $k_{p,r}$ has *no* head dimension (the lightning indexer uses a single shared key projection across all $H=64$ query heads) so the same $k_{p,r}$ multiplies every query head j . The kernel applies the per-token FP8 scale $\text{scale}_{p,r}$ *after* the cross-head sum, which is mathematically equivalent to dequantising first because $\text{scale}_{p,r}$ is per- (p,r) , not per-element, and lets the cross-head reduction read straight from the FP8 tl.dot accumulator. Padded positions ($t \geq \text{seq_lens}[b]$) are written as $-\infty$ so the radix top- K skips them.

Indices.

- $b \in [0, B)$: batch
- $p \in [0, P_{\text{max}})$: logical KV page (stride-64 partition of the KV sequence)

- $r \in [0, 64)$: offset within page; $t = 64p + r$ is the global token position written into the score buffer $I \in \mathbb{R}^{B \times \text{max_seq_padded}}$
- $j \in [0, H)$: query head ($H = 64$); $k_{p,r}$ has no head index

Precision.

- $q_b^{(j)} \in \mathbb{F}_{8, \text{e4m3}}^{128}$: query embedding (FP8), per-head/batch
- $k_{p,r} \in \mathbb{F}_{8, \text{e4m3}}^{128}$: key from cache (FP8), per-page, per-row (shared across heads)
- $w_b^{(j)} \in \mathbb{R}$: learned head weight (float32)
- $sc_{p,r} \in \mathbb{R}$: per-(page, row) FP8 dequant scale (float32)
- Score output $I_{b,t} \in \mathbb{R}$ (bf16 in storage; fp32 in kernel)

Design.

- One program per (b, p) processes one page's 64 tokens.
- $\text{num_warps}=2$ (64 threads/block) matches the B200 tcgen05 64×64×128 MMA tile.
- 32 blocks/SM fits the B200 block-count cap $\rightarrow$ single wave across grid.
- ReLU clips negative dot products before the weighted head sum; unlike softmax, low-relevance keys contribute nothing rather than a small positive probability.

3.2 Top-K Selection: Radix + Page-Table Transform

After scoring all tokens, run a single-CTA filtered radix top-K over each batch row of I to select the $K=2048$ highest-scoring *logical* positions t (in $[0, \text{max_seq_padded})$):

$$\{t_{b,1}, \dots, t_{b,K}\} = \text{RadixSelect}(I_b, K) \quad (2)$$

Each selected logical position t is then **remapped** to the global token id via the per-batch `block_table`:

$$\text{token_id}_{b,k} = \text{block_table}[b, \lfloor t_{b,k}/64 \rfloor] \cdot 64 + (t_{b,k} \bmod 64) \quad (3)$$

The remap is fused inside the radix CTA's emit phase; the kernel writes `token_id` directly to the destination tensor in a single store, with no intermediate buffer of logical indices.

The radix top-K kernel employs a two-stage filtered approach that avoids the typical multi-pass GMEM bottleneck:

Stage 1: Coarse Histogram (1 KB, fully in SMEM)

- Accumulate histogram over 256 coarse bins (upper 8 bits of `bf16`)
- Scan to find threshold bin: where cumulative count $> k$
- Collect all items in bins $>$ threshold_bin

Stage 2: Threshold Bin Refinement (24 KB SMEM buffer per stage, 48 KB total dynamic)

- Buffer all threshold-bin candidates in SMEM. Each buffer holds 6K integers (24KB); a double-buffered design uses 48KB of dynamic SMEM per block. The bound is sized so DSA's empirical $\text{max_seq_padded} \leq 5824 < 6144$ candidates always fit.
- Use lower 8 bits of `bf16` for fine filtering within the threshold bin
- Single refinement pass to extract exact top- k items and apply page-table transform

Why this is fast: Traditional radix top-K requires 3–4 GMEM passes for 32-bit floats, scanning the entire input multiple times. This kernel short-circuits after the coarse pass: all candidates that matter fit in SMEM, so refinement stays in SMEM. The 24 KB per-buffer sizing is safe for DSA ($\text{max_seq_padded} \leq 5824 < 6144$), unlike the generic radix which must handle unbounded inputs and risk overflow. We also dispatch threads-per-block at runtime: $\text{BT}=512$ for $\text{max_len} \leq 4096$ (more CTAs per SM), $\text{BT}=1024$ above (more compute per CTA).

4 Sparse Attention Kernel

4.1 Architecture: Split-K Fused Attention

The sparse attention kernel computes per-head attention over only the top-K key/value tokens selected by the indexer. For query token t and head $h \in [0, H)$ ($H=16$):

$$o_{t,h} = \sum_{s \in s_i[t]} \text{softmax}_s(\langle q_{t,h}, k_s \rangle / \sqrt{d}) \cdot v_s,$$

where $s_i[t] \subseteq \{0, \dots, S-1\} \cup \{-1\}$ is the size- K sparse-index list emitted by the indexer (positions $s = -1$ are padding and are masked out of the softmax). S is the KV-cache token count for this batch element.

The kernel uses the Multi-head Latent Attention (MLA) convention: $q_{t,h} = [q_{t,h}^{\text{nope}}, q_{t,h}^{\text{pe}}] \in \mathbb{R}^{D_{\text{ckv}}+D_{\text{kpe}}}$, $k_s = [\text{ckv}_s, k_s^{\text{pe}}] \in \mathbb{R}^{D_{\text{ckv}}+D_{\text{kpe}}}$, and $v_s = \text{ckv}_s \in \mathbb{R}^{D_{\text{ckv}}}$ (value is the CKV latent, shared across heads; only the query is per-head). $d = D_{\text{ckv}} + D_{\text{kpe}} = 576$ is the full inner-product dimension; the framework supplies $\sqrt{d}^{-1}$ as the `sm_scale` parameter, so the kernel does not derive it internally. Inside the kernel, the inner product is split as $\langle q_{t,h}^{\text{nope}}, \text{ckv}_s \rangle + \langle q_{t,h}^{\text{pe}}, k_s^{\text{pe}} \rangle$ and accumulated by two back-to-back `t1.dot` calls so the CKV and PE blocks are paged into registers separately.

4.1.1 Dispatch Strategy. The kernel branches on query length T (number of tokens):

Table 1: Dispatch strategy for the *Sparse Attention* kernel.

Query Length	Strategy
$T \leq 2$	D-parallel fused (no split)
$T \geq 3$	Split-K with atomic barrier

4.1.2 Split-K Kernel ($T \geq 3$). Two-phase execution on a single grid launch:

- (1) **Phase 1 (Split):** Each $(\text{token_id}, \text{split_id})$ program processes one split of the sparse-index list. With $J = \text{NUM_SPLITS}$ splits, split j owns the *stride- J* subset of $s_i[t]$:

$$s_i^{(j)}[t] = \{s_i[t][j + iJ] : i = 0, \dots, K/J - 1\},$$

so each split visits K/J keys and the union over j recovers the full top-K list. The stride pattern (instead of a contiguous $[jK/J, (j+1)K/J]$ slice) spreads the prefix-heavy valid positions evenly across split. Contiguous splits would dump all valid keys onto split 0 on workloads where the

Table 2: Runtime for the Top- K kernel.

Path	Workloads	Mean Latency
Passthrough	69	$\sim 2.0 \mu\text{s}$
Score + Radix	59	$\sim 13.2 \mu\text{s}$
Overall	128	$\sim 7.2 \mu\text{s}$

index list is short. Each split runs FlashAttention’s online softmax over its own subset and buffers the partial state $(m_j, \ell_j, \text{acc}_j)$ to GMEM.

- (2) **Phase 2 (Combine):** After every split for this token has completed (enforced by the per-token atomic-counter barrier), each $(\text{token_id}, \text{d_slice_id})$ program combines all J partial states along its D -slice using the standard online-softmax merge:

$$m = \max_j m_j, \quad \ell = \sum_j \ell_j e^{m_j - m}, \quad \text{out} = \left(\sum_j \text{acc}_j e^{m_j - m} \right) / \ell.$$

The kernel actually runs this in $\log_2$ space ($e^{m_j - m}$ implemented as $\exp 2(m_j - m)$, with logits pre-scaled by $\sqrt{d}^{-1} \cdot \log_2 e$ so the softmax base change cancels): mathematically identical to the equation above, faster on hardware. The combine emits the final $o_{t,h}[\text{d_slice}]$ slice and, on $\text{d_slice_id} == 0$, the log-sum-exp $\text{lse}_{t,h} = m + \log_2 \ell$ used by downstream pipelines.

4.1.3 Small- T Kernel ($T \leq 2$). For very small T , a separate D-parallel kernel avoids split overhead. Each program handles one $(\text{token}, \text{d_slice})$ pair, processing all sparse keys for that query token in a single fused kernel. No intermediate buffering; output directly to GMEM.

4.2 Implementation Details

- **Partial Accumulator:** Using bfloat16 for intermediate weighted sums reduces register pressure and L2 bandwidth vs. float32.
- **Per-T Scratch Buffers:** Split results are buffered in pre-allocated GMEM per $(T, \text{batch_size})$ pair.
- **Atomic Barrier:** Phase 1 programs increment a per-token counter; Phase 2 programs spin until counter reaches NUM_SPLITS, then proceed.
- **num_warps=4:** Each program block uses 4 warps (128 threads). Sweeps at $\{2, 4, 8, 16\}$ on B200 picked 4 as the best mean across the 23 workloads; we did not derive this from first principles.

5 Performance Analysis

Top- K

We profiled all 128 indexer workloads on B200 to verify single-wave grid placement. The contest indexer workloads span a range of batch sizes and paged KV cache depths. The grid size is computed as $G = B \times P_{\max}$ where B is batch size and $P_{\max}$ is the maximum number of logical pages (stride-64 partitions).

B200 has 148 SMs and a block-count cap of 32 blocks/SM [4]. Max concurrent blocks = $32 \times 148 = 4736$.

Table 3: Parameter ranges for the Top- K kernel workload.

Parameter	Range
Batch size	1 to 31
Max pages (per batch)	1 to 91
Grid size (blocks)	1 to 2730
$\text{max_seq_padded} = P_{\max} \times 64$	64 to 5824

Occupancy Results. across the 128 workload shapes:

- **Max grid size:** $30 \times 91 = 2730$ blocks
- **Single-wave placement:** $128/128 = 100\%$ (max grid $2730 \leq 4736$)
- **Max utilization:** $2730/4736 \approx 57.6\%$
- **Headroom on the largest shape:** $4736 - 2730 = 2006$ blocks

All 128 workloads fit in a single wave. The largest shape saturates over half of B200’s block-count cap; smaller shapes leave more headroom and finish faster.

Kernel Configuration.

- 64 threads = 2 warps, minimal register pressure
- Matches B200’s tcgen05 $64 \times 64 \times 128$ MMA tile
- Enables full 32-block/SM occupancy across all workload shapes

Table 4: Score-kernel num_warps sweep on Modal B200, mean over all 128 workloads.

Warps	Threads	Blocks/SM	No-Skip	All
2	64	32	13.2 μs	7.2 μs
4	128	16	18.0 μs	9.4 μs
8	256	8	17.9 μs	9.3 μs

At $\text{num_warps}=2$, $32 \text{ blocks/SM} \times 148 \text{ SMs} = 4736$ concurrent blocks, so the largest grid (2730) clears in a single wave. Doubling the threads/block to 128 halves the block-count headroom and pushes the largest shapes into a second wave; the latency cost is visible. The MMA tile sizes implied by each num_warps (Blackwell tcgen05 $64 \times 64 \times 128$ at $\text{num_warps}=2$) are consistent with the natural fit, but the dominant effect is occupancy, not tile shape.

Sparse Attention

We have done hyper-parameter sweep for the split-K kernel against all 23 workloads on Modal B200 (Table 6). Our choices for those parameters are mostly empirical.

Roofline analysis

Three patterns drive the under-utilisation. **Sparse attention** is L1-bound. **Indexer hard** is dominated by kernel launch and the radix top- K kernel that runs after the score phase: one CTA per batch row, serial within each CTA. **Indexer easy** is trivially small, the gap is from the kernel launch.

Table 6: Sparse-attention hyper-parameter sweep on B200.

Configuration	Small-T	Split-K	Overall
num_warps=2	9.3 μ s	18.6 μ s	15.0 μ s
num_warps=4	7.2 μs	10.8 μs	9.4 μs
num_warps=8	7.9 μ s	11.9 μ s	10.4 μ s
num_warps=16	10.8 μ s	14.6 μ s	13.1 μ s
NUM_SPLITS=4	7.4 μ s	19.4 μ s	14.7 μ s
NUM_SPLITS=8	7.3 μ s	13.2 μ s	10.9 μ s
NUM_SPLITS=16	7.2 μs	10.8 μs	9.4 μs
NUM_SPLITS=32	9.3 μ s	runtime error [†]	—

[†] BLOCK_N=128 > SPLIT_SIZE=2048/32=64; out-of-range gather.

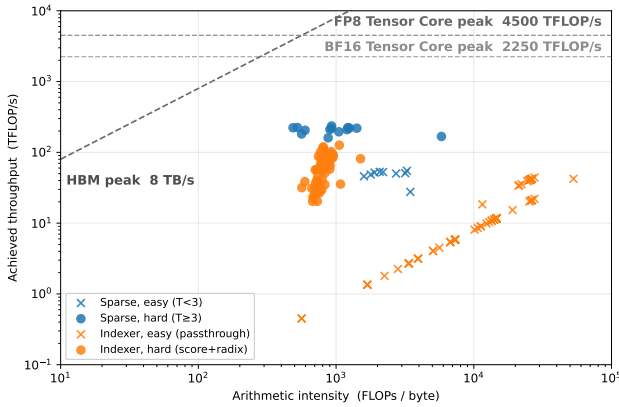


Figure 1: Roofline on B200 on Modal, built from Nsight Compute counters.

6 Human vs. Agent Contributions

6.1 Kernel Bootstrapping

We did not start writing the kernel by hand or with an autonomous loop. We started with a few-shot conversational prompt that asked the agent to produce a first reasonably-optimized Triton kernel. From there a human read the code closely, tile shapes, SM utilization, redundant computation, padding handling, and proposed ideas or next steps to the agent. Such as dispatching different shapes for different workloads, trying warp specialization, split-K, and so on. An autonomous loop was *built* from the patterns that emerged from those conversations, without this initial bootstrap phase, we would not have known how to build a good agent or a good kernel.

6.2 Failure modes the human kept correcting

Two patterns dominated. First, the agent often declared a technique *infeasible* when in reality its implementation was wrong; the human had to repeatedly contradict the agent’s confidence and point at a specific bug rather than at the strategy. Second, the agent fell into *partial implementations* of correct strategies: when reminded to skip top- k on the trivial sub-distribution, it would skip the sort but still do the FP8 scoring, or still pre-fill the output with -1 from a separate launch. Each correction was small and repeatedly correctable, but they only ended when the human said “stop, that’s still doing useless work, remove it.”

6.3 Workload-aware reasoning

The largest single improvement on the indexer came from an observation about the *workload*, not the kernel: roughly half of the contest traces have a valid token count that fits inside $K=2048$, collapsing top- k selection to a structural mapping. The human noticed this pattern while reading the trace metadata. We then reran the autonomous loop multiple times to see whether it would re-discover the optimization on its own. It did not. The pattern repeated across runs: the agent reasoned over the kernel’s code, not over the kernel’s inputs. This led us to create a specialized agent for analyzing workload distribution.

6.4 Improving Fully AI-Generated Kernels

After we had collected enough experience on how to use AI tools and how to guide them to build kernels autonomously, we ran the loop overnight. To our surprise, it generated kernels faster than the human-agent collaboration kernels. The human contributed very little code in the final kernel, mostly acting as a reviewer. The agent was able to come up with a very well-optimized kernel on its own. We kept the autonomous loop’s Triton sparse attention and FP8 scoring, while replacing Top-K with the collaboration phase’s CUDA FilteredTopK. We also ran one final autonomous pass purely for hyper-parameter sweeps (tile shapes, NUM_SPLITS, num_warps, num_stages).

Final headline numbers. The Sparse Attention kernel achieves an average runtime of 0.009 ms and the TopK indexer 0.009 ms, yielding an average 37.07 $\times$ speedup over the FlashInfer baselines and ranking our kernel first in the competition.

Acknowledgments

We thank the MLSys 2026 contest organizers at NVIDIA and the FlashInfer team for the evaluation harness and traces, the DeepSeek team for releasing DSA, and the Modal team for the remote-GPU backend used for every measurement in this paper.

Table 5: Per-category NCU utilisation on B200 (--set roofline averages).

Category	Active SMs	L1/TEX	L2	DRAM	Compute
Sparse, easy ($T < 3$)	1–2 / 148	42 %	0.9 %	0.3 %	2.2 %
Sparse, hard ($T \geq 3$)	6–8 / 148	38 %	3.8 %	3.0 %	9.2 %
Indexer, easy (passthrough)	1–30 / 148	4.4 %	0.5 %	0.0 %	0.3 %
Indexer, hard (score+radix)	5–86 / 148	9.9 %	0.8 %	0.9 %	1.4 %

References

- [1] FlashInfer team. *MLSys 2026 FlashInfer AI Kernel Generation Contest*. <https://mlsys26.flashinfer.ai/>, 2026.
- [2] DeepSeek-AI. *DeepSeek-V3.2: Pushing the Frontier of Open Large Language Models* 2025.
- [3] Tillet, Philippe and Kung, Hsiang-Tsung and Cox, David. *Triton: an intermediate language and compiler for tiled neural network computations*. Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages, 2019.
- [4] NVIDIA. *Blackwell Tuning Guide* (CUDA 13.2). <https://docs.nvidia.com/cuda/blackwell-tuning-guide/index.html>, 2025.